TFT Living Room Display — Project Documentation

Version: 0.10 **Date:** April 2026 **Status:** Phases 1, 2 and 3 complete

---

1. Project Overview

A wall-mounted smart display for the living room built around an ESP32 microcontroller and a 3.2” TFT touchscreen. The display serves three purposes:

- **Dashboard mode** — shows the current state of living room lights and heating, with touch controls to turn them on and off
- **Screensaver mode** — after 60 seconds idle, switches to a photo slideshow pulling images from the home NAS
- **Doorbell mode** — when the Eufy doorbell is pressed, the display immediately shows “Image loading…” then switches to a live snapshot from the front door camera

---

2. Hardware

Component	Details
Microcontroller	ESP32 WROOM-32 (30-pin DevKit, Type-C USB)
Display	3.2” SPI TFT, ILI9341 driver, 240×320 resolution
Screen SKU	MSP3218
Touch	Resistive (XPT2046 controller) — stylus included; large buttons work with finger
Connection	Dupont jumper wires

2.1 Wiring

Screen Pin	ESP32 Pin	Notes
VCC	3.3V	
GND	GND	
CS	GPIO 5	Display chip select
RESET	GPIO 4	
DC	GPIO 16	**Note: GPIO 2 conflicts with boot — use GPIO 16**
SDI (MOSI)	GPIO 23	
SCK	GPIO 18	Shared with T_CLK
LED	VIN (5V)	Backlight — 5V gives good brightness
SDO (MISO)	GPIO 19	Shared with T_DO
T_CLK	GPIO 18	Shared with SCK
T_CS	GPIO 15	Touch chip select
T_DIN	GPIO 23	Shared with MOSI
T_DO	GPIO 19	Shared with MISO
T_IRQ	GPIO 21	Touch interrupt

Important: GPIO 2 on the ESP32 WROOM-32 is connected to the onboard LED and causes display initialisation to fail if used as the DC pin. Always use GPIO 16 for DC.

Shared SPI pins: SCK/T_CLK, MOSI/T_DIN, and MISO/T_DO all share the same ESP32 pins. Connect both screen wires to the same ESP32 pin (twist wire ends together into one Dupont socket).

---

3. System Architecture

```
+---------------------+     WiFi / REST API    +----------------------+
|   ESP32 + TFT       | <--------------------> |  Home Assistant      |
|   Living Room       |                        |  Raspberry Pi 5      |
|   Display           |     WiFi / HTTP        |  192.168.0.38:8123   |
|   192.168.0.103     | <--------------------> +----------------------+
|                     |                        |  NAS Photo Server    |
|                     |     WiFi / MQTT        |  Docker on DPX2800   |
|                     | <--------------------> |  192.168.0.248:5000  |
+---------------------+  (doorbell trigger)    +----------------------+
                                                         ^
                                                         | WebSocket
                                                         | ws://192.168.0.38:3000
                                                +--------+-------------+
                                                | eufy-security-ws     |
                                                | on Home Assistant Pi |
                                                | (Eufy local hub)     |
                                                +----------------------+
```

Doorbell flow

1. Doorbell button pressed
2. Eufy hub notifies Home Assistant via local integration
3. HA automation fires an MQTT message to topic `tft/doorbell`
4. ESP32 receives MQTT message — immediately shows “Image loading…” on screen
5. ESP32 sends HTTP GET to `/doorbell-snapshot` on the NAS (blocks waiting)
6. NAS WebSocket listener (connected to eufy-security-ws on port 3000) receives the picture event
7. NAS processes the image (crop top half, resize to 320×240) and signals ready
8. NAS responds to ESP32 with the JPEG — image displayed
9. After `DOORBELL_TIMEOUT_MS` (60s) or touch, display returns to previous mode
10. If no image arrives within 60s, NAS returns an error and ESP32 shows “Image not available” for 10s

---

4. Software — Development Environment

4.1 Mac Toolchain

Tool	Version	How installed
Git	2.50.1	Apple built-in
Homebrew	5.1.1	brew.sh
Python 3	3.14.3	Homebrew
Node.js	25.8.1	Homebrew
VS Code	1.113.0	`brew install --cask visual-studio-code`
GitHub CLI	2.89.0	`brew install gh`
PlatformIO	latest	VS Code extension

Tool	Version	How installed
Pandoc	3.9.0	`brew install pandoc`

Run `scripts/setup-macbook.sh` on a new Mac to install and verify all tools automatically.

4.2 Project Repository

- **GitHub:** https://github.com/dpbrighton/TFTDisplay
- **Local path (Mac mini):** `~/Documents/Claud/TFTDisplay`

4.3 Version Snapshots

Tag	Description
v0.1-project-start	Initial structure and README
v0.2-firmware-skeleton	PlatformIO project, config, basic WiFi + display
v0.3-display-working	Fixed DC pin (GPIO 16), display confirmed working
v0.4-dashboard-working	Phase 1 complete — HA dashboard live
v0.5-dashboard-refined	No-flicker refresh, larger heating buttons
v0.6-photo-server-ui	NAS photo server with web UI for folder selection
v0.7-screensaver-working	Phase 2 complete — photo screensaver
v0.8-photo-orientation	EXIF orientation fix, photo streaming improvements
v0.9-doorbell-working	Phase 3 complete — Eufy doorbell integration
v0.10-photo-freeze-fix	Non-blocking photo retry, stuck-photo auto-recovery, hardware watchdog

---

5. ESP32 Firmware

5.1 Build Tool

PlatformIO (VS Code extension). Open the `firmware/` folder in VS Code. Use the tick button to build and arrow to upload.

5.2 Libraries

Library	Purpose
bodmer/TFT_eSPI	Display and touch driver
bblanchon/ArduinoJson	Parsing Home Assistant JSON responses
knolleary/PubSubClient	MQTT — receives doorbell ring trigger
bodmer/TJpg_Decoder	JPEG decoding for photo slideshow and doorbell image

5.3 Configuration

Edit `firmware/include/config.h` before flashing. This file is excluded from Git (credentials). Use `config.h.template` as a starting point on a new machine.

Setting	Description
`WIFI_SSID` / `WIFI_PASSWORD`	Home WiFi credentials

Setting	Description
`HA_HOST`	Home Assistant IP (192.168.0.38)
`HA_PORT`	8123
`HA_TOKEN`	Long-lived access token from HA profile
`PHOTO_SERVER_HOST`	NAS IP (192.168.0.248)
`PHOTO_SERVER_PORT`	5000
`MQTT_HOST`	MQTT broker IP (same as HA host)
`MQTT_PORT`	1883
`MQTT_USER` / `MQTT_PASS`	MQTT broker credentials
`MQTT_CLIENT_ID`	`tft-display`
`MQTT_TOPIC_DOORBELL`	`tft/doorbell`
`DASHBOARD_POLL_MS`	How often to refresh HA state (default 10000ms)
`SCREENSAVER_TIMEOUT`	Idle time before screensaver (default 60000ms)
`DOORBELL_TIMEOUT_MS`	How long to show doorbell image (default 60000ms)

5.4 Key Implementation Notes

- **WiFi modem sleep disabled** — `WiFi.setSleep(false)` is called before `WiFi.begin()`. Without this, the ESP32 enters modem sleep between packets, causing partial TCP reads at multiples of 1436 bytes (TCP MSS). This was the root cause of photo slideshow stalls.
- **HTTP connection reuse disabled** — `http.setReuse(false)` + `Connection: close` header prevents stale keep-alive connections returning 0 bytes on reuse.
- **Manual read loop** — photo and doorbell image reads use a manual while-loop with a hard deadline rather than `readBytes()`, allowing `mqttClient.loop()` to be called during the read so MQTT stays alive.
- **Doorbell priority** — the `doorbellTriggered` flag is checked inside the photo read loop so a ring always interrupts a photo download immediately.
- **Doorbell timer reset** — `doorbellTime` is reset to `millis()` when the image is successfully displayed, so the user gets the full `DOORBELL_TIMEOUT_MS` view time from when the image appears, not from when the bell rang.
- **Non-blocking photo retry** — `showNextPhoto()` does not block on failure. Instead of `delay(2000)` + second attempt, a failed fetch sets `lastPhotoTime` 3 seconds behind so the loop's own timer triggers the retry naturally. Touch stays fully responsive during the wait.
- **Stuck-photo auto-recovery** — `lastPhotoSuccess` tracks the last time a photo was successfully drawn. If no successful photo has been shown in 60 seconds (`PHOTO_STUCK_MS`), the loop calls `checkWifi()` and forces an immediate retry. This handles cases where the NAS is unreachable or the TCP stack silently stalls.
- **Hardware watchdog** — `esp_task_wdt_init(30, true)` initialises a 30-second hardware watchdog in `setup()`. `esp_task_wdt_reset()` is called at the top of every `loop()` iteration and inside the photo read loop. If the loop ever genuinely hard-stalls, the device reboots cleanly.

5.5 Touch Calibration

On first boot, the display runs a touch calibration routine — touch each marker with the stylus. Calibration data is saved to the ESP32's non-volatile storage and is not repeated unless the device is erased.

To force recalibration: in PlatformIO, erase the flash (`pio run -t erase`) then reflash.

---

6. Home Assistant Setup

6.1 Entities Used

Entity ID	Description
`light.drawing_room_light_1`	Drawing room overhead light 1
`light.drawing_room_light_2`	Drawing room overhead light 2
`light.drawing_room_light_3`	Drawing room overhead light 3
`light.drawing_room_light_4`	Drawing room overhead light 4
`light.old_lamp`	Living room lamp (smart bulb)
`switch.living_room_lamp_socket_1`	Living room lamp socket
`climate.living_room`	Living room TRV (Tado)
`binary_sensor.front_door_bell_ringing`	Eufy doorbell ring sensor

6.2 Doorbell Automation

Trigger: `binary_sensor.front_door_bell_ringing` state → `on`

Action: publish MQTT message - Topic: `tft/doorbell` - Payload: `ring` - QoS: 1

This is the only HA involvement in the doorbell flow. Image retrieval is handled entirely between the NAS and eufy-security-ws — HA is not used for image delivery.

6.3 MQTT Broker

Mosquitto add-on running on the HA Raspberry Pi (192.168.0.38:1883). User `mqtt-user` with appropriate password set in the Mosquitto config.

---

7. NAS Photo Server

7.1 Location

- **NAS:** UGREEN DXP2800 at `192.168.0.248`
- **Docker management:** UGREEN Docker (built-in) — SSH to NAS for rebuilds
- **Source files on NAS:** `/volume1/docker/nas-server/`
- **Config folder:** `/volume1/docker/tft-config/`
- **Photos root:** `/volume1/Photos`

7.2 Docker Setup

The server runs as a Docker container. Project defined in `/volume1/docker/nas-server/docker-compose.yml`.

Volumes mounted into the container:

Host path	Container path	Access
`/volume1/Photos`	`/photos`	Read-only
`/volume1/docker/tft-config`	`/config`	Read-write (saves config)

> **Note:** `app.py` and `requirements.txt` are baked into the Docker image at build time. To deploy code changes, upload the file via UGREEN file manager then rebuild:

```
ssh Davidadmin@192.168.0.248
cd /volume1/docker/nas-server
docker compose build --no-cache && docker compose up -d
```

7.3 Python Dependencies

`requirements.txt`:

```
flask==3.1.0
pillow==11.2.1
websocket-client==1.8.0
```

`websocket-client` is required for the eufy-security-ws WebSocket listener. If it is missing from the image, the container logs will show `websocket-client not available — eufy WebSocket listener disabled` and doorbell images will fall back to HA fetch only.

7.4 eufy-security-ws WebSocket Listener

On startup, the NAS server connects to `ws://192.168.0.38:3000` (the eufy-security-ws add-on running on the HA Pi) and sends a `start_listening` command. It then listens for `property changed` events with `name == "picture"` from the T8214 doorbell device.

When a picture event arrives: 1. Image bytes extracted from the Node.js Buffer payload 2. Image cropped to top half (T8214 is a composite 640×880 image — top half is the main front camera) 3. Resized to 320×240 and saved as JPEG 4. Cached in memory and a threading Event flag set

The `/doorbell-snapshot` endpoint blocks waiting for this flag (up to 60s), then serves the cached image and clears the cache. The next ring starts with a clean slate.

Connection drops are handled automatically — the listener thread reconnects every 15 seconds on failure.

eufy-security-ws add-on configuration (HA Pi): The station must be added via the add-on UI with `serial_number: T8030T23244602B4` and `ip_address: 192.168.0.129` (DHCP reservation set in router). This forces local P2P rather than cloud relay. Without this, the P2P client has no LAN IP and falls back to AWS relay servers, adding further latency. The `event_duration` config option controls how long binary sensors (e.g. `ringing`) stay triggered after an event — it does not affect snapshot timing.

7.5 API Endpoints

Endpoint	Method	Description
`/`	GET	Web UI for folder selection and settings
`/next-photo`	GET	Returns a random photo as 320×240 JPEG
`/doorbell-snapshot`	GET	Blocks until fresh doorbell image ready (max 60s), returns JPEG
`/eufy-ws-status`	GET	Diagnostic: WebSocket connection state and cache info
`/settings`	GET	Returns `{"display_seconds": N}`
`/health`	GET	Returns photo count and status
`/save-config`	POST	Saves new config (called by web UI)
`/test-doorbell`	GET	Diagnostic: tests HA image_proxy connection

7.6 Configuration Web UI

Open `http://192.168.0.248:5000` in any browser on the local network to: - Tick/untick photo folders to include - Set seconds per photo - Toggle shuffle and include-subfolders - Set HA host, port and token (needed for doorbell fallback) - Preview a photo or doorbell snapshot

7.7 Photo Orientation

The server automatically corrects photos that have EXIF orientation metadata (e.g. photos taken on a modern phone held in portrait). This is handled by `ImageOps.exif_transpose()` in `app.py`.

For older photos and scans with no EXIF orientation data, two one-off utility scripts are provided in `tools/`:

Script	Purpose
`audit_photo_orientation.py`	Scans the photo library and reports EXIF orientation status
`fix_photo_orientation.py`	Uses face detection (OpenCV) to fix orientation of suspect photos

To run these on a new machine:

```
python3 -m venv ~/photo-tools-venv
source ~/photo-tools-venv/bin/activate
pip install pillow opencv-python piexif

# Audit first
python3 tools/audit_photo_orientation.py /Volumes/Photos

# Dry run fix (no files changed)
python3 tools/fix_photo_orientation.py /Volumes/Photos

# Apply fixes
python3 tools/fix_photo_orientation.py /Volumes/Photos --apply
```

> The NAS must be mounted at `/Volumes/Photos` via SMB before running these scripts.

---

8. Dashboard Operation

Area	Function
Top bar	“LIVING ROOM” title + WiFi status dot (green = connected)
Left half	Drawing room lights — ON / OFF buttons, status dot
Right half	Lamps — ON / OFF buttons, status dot
Bottom strip	Heating — current temp (large), target temp, – and + buttons

- Dashboard polls Home Assistant every 10 seconds
- Only changed sections are redrawn (no flicker)
- Temperature changes made in the Tado app take 1–2 minutes to appear (Tado → HA sync delay)

---

9. Screensaver Operation

- After **60 seconds** with no touch, the display switches to photo slideshow
- Photos are fetched from the NAS server, pre-resized to 320×240
- Display duration per photo is set in the NAS web UI
- **Touch anywhere** to return to the dashboard immediately

---

10. Doorbell Operation

- When the doorbell rings, the ESP32 receives an MQTT message instantly
- Display switches to a black screen with a red “FRONT DOOR” header and “Image loading…” text
- The NAS blocks waiting for the eufy-security-ws picture event (up to 60 seconds)
- When the image arrives it is sent to the ESP32 and displayed full-screen with the header overlay
- The display auto-dismisses after 60 seconds, or can be dismissed by touching the screen
- If no image arrives within 60 seconds, “Image not available” is shown for 10 seconds, then the display returns to its previous mode (dashboard or screensaver)

Note on image timing: The snapshot typically arrives approximately 60 seconds after the bell is pressed. This is a limitation of the eufy-security-ws library (v2.1.0), which uses a hardcoded 60-second delay after receiving the ring event before querying the hub database for the snapshot — presumably to allow the hub time to write the file to disk. The P2P connection between the HA Pi and the Eufy hub is local (192.168.0.129), confirmed via debug logging. Reducing this delay would require patching the eufy-security-client library.

---

11. Known Issues / Notes

- **Eufy image latency:** Doorbell snapshots take approximately 60 seconds to arrive. This is a hardcoded delay in the eufy-security-client library (v2.1.0) — after receiving the ring event it waits ~60 seconds before querying the hub database for the snapshot. The P2P connection is local (192.168.0.129 — confirmed). The display shows “Image loading…” during this time. Reducing the delay would require patching the library.
- **Tado sync lag:** Temperature changes in Tado app take 1–2 minutes to reflect on the display. This is a Tado–HA sync limitation.
- **Resistive touch:** Large buttons work well with a finger. For precise input use the supplied stylus.
- **Photo server startup:** On NAS reboot, the container scans all selected photo directories before serving. With 18,000+ photos this takes a few seconds.
- **Portrait photos (old/scanned):** Photos taken on modern phones are auto-corrected via EXIF orientation. Older scanned photos without EXIF data were corrected using the `tools/fix_photo_orientation.py` script. ~167 photos were fixed in the initial run.
- **NAS Docker rebuilds:** Because `app.py` is baked into the image, any code change requires an SSH rebuild (not just a container restart via the UGREEN UI).
- **Photo screensaver freeze (resolved in v0.10):** The screensaver occasionally stopped cycling photos and only recovered when the screen was touched. Root causes: a blocking `delay(2000)` in `showNextPhoto()` prevented touch input during retries; HTTP timeouts of 8s were too generous for a local JPEG; no automatic recovery if the NAS was silently unreachable. Fixed in v0.10 — see implementation notes above.
- **Doorbell watchdog reboot (resolved in v0.10):** After adding the hardware watchdog in v0.10, ringing the doorbell caused an immediate reboot. The doorbell image fetch intentionally blocks for up to 65s waiting for the NAS snapshot — longer than the 30s watchdog window. Fixed by unregistering the main task from the watchdog before the fetch and re-registering immediately after.